# Experiment 1

**Date : 12-01-2026**

# Principal Component Analysis & Exploratory Data Analysis

## Description :

Principal Component Analysis is a statistical method that converts a set of correlated variables to a set of uncorrelated variables through orthogonal transformations. It is an unsupervised learning algorithm that is used for dimensionality reduction in machine learning. The transformed features are called principal components.

## Algorithm :

Step 1: Standardize the dataset.

Step 2: Calculate the covariance matrix for the features in the dataset.

Step 3: Calculate the eigenvalues and eigenvectors for the covariance matrix.

Step 4: Sort eigenvalues and their corresponding eigenvectors.

Step 5: Pick k eigenvalues and form a matrix of eigenvectors.

Step 6: Transform the original matrix.

## Dataset :

https://archive.ics.uci.edu/ml/machine-learning-databases/balance-scale/balance-scale.data

## Dataset Information :

This data set was generated to model psychological experimental results. Each example is classified as having the balance scale tip to the right, tip to the left, or be balanced. The attributes are the left weight, the left distance, the right weight, and the right distance. The correct way to find the class is the greater of (left-distance * left-weight) and (right-distance * right-weight). If they are equal, it is balanced.

No.of instances: 625

No.of attributes: 4

## Dataset Information :

1. Class Name: 3 (L, B, R)
2. Left-Weight: 5 (1, 2, 3, 4, 5)
3. Left-Distance: 5 (1, 2, 3, 4, 5)
4. Right-Weight: 5 (1, 2, 3, 4, 5)
5. Right-Distance: 5 (1, 2, 3, 4, 5)

# Ex - 1(a) - Principal Component Analysis(PCA)

## Code - PCS :

PCA.ipynb × | *https\archive.ics.uci.edu\ml\machine-learning-databases\balance-scale\balance-scale.data.txt* | BLACKBOX

ML_Lab > Ex_1(PCA) > PCA.ipynb > M↓ Step-1 - Load the dataset > # read url(The URL must be a string with full slashes)

Generate + Code + Markdown | Run All | Restart | Clear All Outputs | View data ··· | HC_env (Python 3.10.13)

```python
!pip install --upgrade pandas
```

[39] ✓ 2.5s Python

```
Requirement already satisfied: pandas in c:\users\admin\anaconda3\envs\hc_env\lib\site-packages (2.3.3)
Requirement already satisfied: numpy>=1.22.4 in c:\users\admin\anaconda3\envs\hc_env\lib\site-packages (from p
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\admin\anaconda3\envs\hc_env\lib\site-packag
Requirement already satisfied: pytz>=2020.1 in c:\users\admin\anaconda3\envs\hc_env\lib\site-packages (from pa
Requirement already satisfied: tzdata>=2022.7 in c:\users\admin\anaconda3\envs\hc_env\lib\site-packages (from
Requirement already satisfied: six>=1.5 in c:\users\admin\anaconda3\envs\hc_env\lib\site-packages (from pythor
WARNING: Ignoring invalid distribution -atplotlib (c:\users\admin\anaconda3\envs\hc_env\lib\site-packages)
WARNING: Ignoring invalid distribution -atplotlib (c:\users\admin\anaconda3\envs\hc_env\lib\site-packages)
WARNING: Ignoring invalid distribution -atplotlib (c:\users\admin\anaconda3\envs\hc_env\lib\site-packages)
```

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
```

[40] ✓ 0.0s Python

### Step-1 - Load the dataset

```python
# read url(The URL must be a string with full slashes)
# url = " https\archive.ics.uci.edu\ml\machine-learning-databases\balance-scale\balance-scale.data"

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/balance-scale/balance-scale.data"

# Declare the column names(This dataset does not have a header row, so we define column names manually)
col_names = ['Class Name', 'Left-Weight', 'Left-Distance', 'Right-Weight', 'Right-Distance']

#Read the data in the url using read_csv
data = pd.read_csv(url,header=None,names=col_names)

data = pd.DataFrame(data)
print(data.head())
```

[41] ✓ 0.9s Python

```
   Class Name  Left-Weight  Left-Distance  Right-Weight  Right-Distance
0           B            1              1             1               1
1           R            1              1             1               2
2           R            1              1             1               3
3           R            1              1             1               4
4           R            1              1             1               5
```

```python
print(data.shape)
```

[42] ✓ 0.0s Python

```
(625, 5)
```

```python
print(data.info())
```

[43] ✓ 0.0s Python

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 625 entries, 0 to 624
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Class Name      625 non-null    object
 1   Left-Weight     625 non-null    int64
 2   Left-Distance   625 non-null    int64
 3   Right-Weight    625 non-null    int64
 4   Right-Distance  625 non-null    int64
dtypes: int64(4), object(1)
memory usage: 24.5+ KB
None
```

## Step 2 – Preprocess the Dataset

As a next step, let's preprocess the dataset. The features are all on different scales. To bring them all to a common scale, we'll use the StandardScaler that transforms the features to have zero mean and unit variance:

As a next step, let's preprocess the dataset. The features are all on different scales. To bring them all to a common scale, we'll use the StandardScaler that transforms the features to have zero mean and unit variance:

```python
# 'Class Name' is your target (categorical), so we remove it to get our features
X = data.drop('Class Name', axis=1)

# 'Class Name' is kept separately as y (the target we want to predict or label)
y = data['Class Name']
```

[44] ✓ 0.0s Python

```python
# from sklearn.preprocessing import StandardScaler
# std_scaler = StandardScaler()
# scaled_df = std_scaler.fit_transform(df)

# Do with the calculation part also

import numpy as np
import pandas as pd

# 1. Calculate the mean (mu) for each column
# Scikit-learn equivalent: scaler.mean_
mu = np.mean(X, axis=0)

# 2. Calculate the population standard deviation (sigma)
# Must use ddof=0 to match Scikit-learn's default behavior
# Scikit-learn equivalent: scaler.scale_
sigma = np.std(X, axis=0, ddof=0)

# 3. Apply the Z-score formula
# This centers the data (mean=0) and scales it (variance=1)
X_scaled_manual = (X - mu) / sigma

# Check the results
print("Manual Scaled Data:\n", X_scaled_manual.head())
```

Chat (CTRL + I) / Share (CTRL + L)

[45] ✓ 0.0s Python

```
Manual Scaled Data:
   Left-Weight  Left-Distance  Right-Weight  Right-Distance
0    -1.414214      -1.414214     -1.414214       -1.414214
1    -1.414214      -1.414214     -1.414214       -0.707107
2    -1.414214      -1.414214     -1.414214        0.000000
3    -1.414214      -1.414214     -1.414214        0.707107
4    -1.414214      -1.414214     -1.414214        1.414214
```

## Step 3 – Perform PCA on the Preprocessed Dataset

To find the principal components, we can use the PCA class from scikit-learn's decomposition module.

Let's instantiate a PCA object by passing in the number of principal components n_components to the constructor.

The number of principal components is the number of dimensions that you'd like to reduce the feature space to. Here, we set the number of components to 3.

```python
# from sklearn.decomposition import PCA
# pca = PCA(n_components=3)
# pca.fit_transform(scaled_df)

import numpy as np
import pandas as pd

# Assume 'scaled_X' is your standardized data from the previous step
# 1. Compute the Covariance Matrix
# This matrix shows how each pair of features varies together.
# 'rowvar=False' tells numpy that columns are features.
cov_matrix = np.cov(X_scaled_manual, rowvar=False)

# 2. Calculate Eigenvalues and Eigenvectors
# Eigenvectors: The directions of the new axes (Principal Components).
# Eigenvalues: The magnitude of variance in those directions.
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
```

```python
# 3. Sort Eigenvalues and Eigenvectors in descending order
# We want the components that capture the MOST variance first.
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

# 4. Select top 'k' components (e.g., n_components=3)
# We take the first 3 columns of the eigenvector matrix.
k = 3
feature_vector = eigenvectors[:, :k]

# 5. Transform the data (Project onto the new axes)
# We use the dot product to rotate the original data points.
pca_manual = np.dot(X_scaled_manual, feature_vector)

# Create a DataFrame for the result
pca_manual_df = pd.DataFrame(pca_manual, columns=['PC1', 'PC2', 'PC3'])
```

Chat (CTRL + I) / Share (CTRL + L)

✓ 0.0s Python

# Step 4 – Examining Some Useful Attributes of the PCA Object

The PCA instance pca that we created has several useful attributes that help us understand what is going on under the hood.

The attribute components_ stores the directions of maximum variance (the principal components).

```python
# print(pca_manual.components_)

# To match Scikit-learn's .components_ exactly:
components_sklearn_style = feature_vector.T

print("Principal Components (Eigenvectors):")
print(components_sklearn_style)
```

[47] ✓ 0.0s Open 'components_sklearn_style' in Data Wrangler Python

```
Principal Components (Eigenvectors):
[[ 3.96080529e-20 -9.66624081e-02 -2.17369913e-02  9.95079837e-01]
 [ 0.00000000e+00 -7.06124391e-01  7.07751874e-01 -2.18089182e-02]
 [-7.07106781e-01 -4.99305345e-01 -4.99305345e-01 -3.72604936e-02]]
```

```python
# print(sum(pca.explained_variance_ratio_))

print(sum(feature_vector.T))
```

[48] ✓ 0.0s Python

```
[-0.70710678 -1.30209214  0.18670954  0.93601043]
```

# Step 5 – Analyzing the Change in Explained Variance Ratio

We can try running principal component analysis by varying the number of components n_components.

```python
from sklearn.decomposition import PCA
import numpy as np
#nums = np.arange(14)
# Check how many features you actually have
max_features = X_scaled_manual.shape[1]

# Re-define nums to stay within the limit (1 to max_features)
nums = np.arange(1, max_features + 1)
```

[49] ✓ 0.0s Python

```python
var_ratio = []
for num in nums :
    pca = PCA(n_components=num)
    pca.fit(X_scaled_manual)
    var_ratio.append(np.sum(pca.explained_variance_ratio_))

print(f"Variance ratios for 1 to {max_features} components:")
print(var_ratio)
```

Chat (CTRL + I) / Share (CTRL + L)

[50] ✓ 0.0s Open 'var_ratio' in Data Wrangler Python

```
Variance ratios for 1 to 4 components:
[np.float64(0.25), np.float64(0.5), np.float64(0.75), np.float64(1.0)]
```

To visualize the explained_variance_ratio_ for the number of components, let's plot the two quantities as shown:

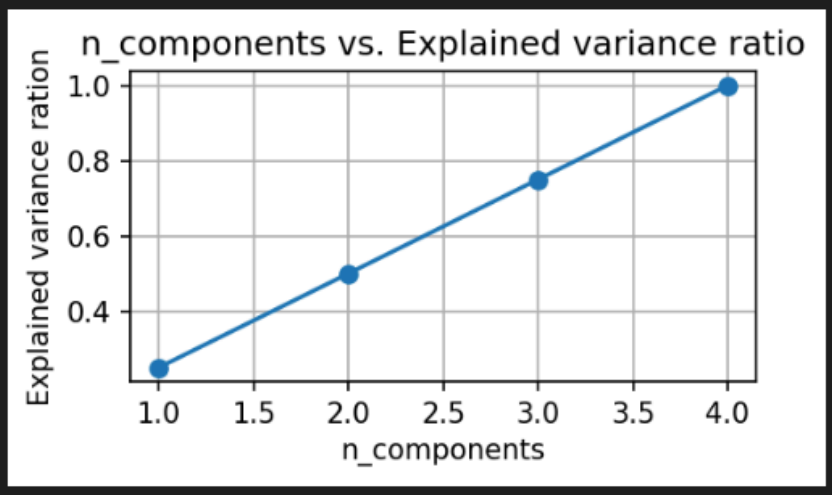
```
import matplotlib.pyplot as plt

plt.figure(figsize=(4,2),dpi=150)
plt.grid()
plt.plot(nums,var_ratio,marker='o')
plt.xlabel("n_components")
plt.ylabel('Explained variance ration')
plt.title('n_components vs. Explained variance ratio')
```

Chat (CTRL + I) / Share (CTRL + L)

[51] ✓ 0.1s Python

```
Text(0.5, 1.0, 'n_components vs. Explained variance ratio')
```




When we use all the 13 components, the explained_variance_ratio_ is 1.0 indicating that we've captured 100% of the variance in the dataset.

In this example, we see that with 6 principal components, we'll be able to capture more than 80% of variance in the input dataset.

**EDA- Data set:**

[https://www.kaggle.com/datasets/benroshan/online-food-delivery-preferencesbangalore-region](https://www.kaggle.com/datasets/benroshan/online-food-delivery-preferencesbangalore-region)

```
anaconda3\envs\hc_env\lib\site-packages)
WARNING: Ignoring invalid distribution -atplotlib (c:\users\admin\
anaconda3\envs\hc_env\lib\site-packages)
```

### Step-1 Import the libraries

```
import io
import sys
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_selection import chi2

from itertools import product
from scipy.stats import chi2_contingency


#Basic libraries
import pandas as pd
import numpy as np

#Visualization libraries
import matplotlib.pyplot as plt
from matplotlib.ticker import FormatStrFormatter
import seaborn as sns
import plotly.express as px
import plotly.graph_objs as go
from plotly import tools
from plotly.offline import iplot

#Geospatial Analysis Libraries
import geopandas as gpd
import math
import folium
from folium import Choropleth, Circle, Marker
from folium.plugins import HeatMap, MarkerCluster
```

### Step-2 Load the Dataset

```
file_name = r'E:\Z_Arise\ML_Lab\Ex_1(b)_EDA\onlinedeliverydata.csv'

data = pd.read_csv(file_name)
data.head()
```

["3","22","Female","Single","Student","No Income","Graduate","6","12.9473","77.5616","560019","Food delivery apps"," Walk-in","Snacks"," Dinner","Veg foods (Breakfast / Lunch / Dinner)"," Bakery items (snacks)","Agree","Agree","Strongly agree","Agree","Strongly agree","Agree","Agree","Agree","Strongly agree","Neutral","Agree","Disagree","Disagree","Neutral","Agree","Agree","Agree","Disagree","Disagree","Neutral","Yes","Anytime (Mon-Sun)","30 minutes","Disagree","Agree","Agree","Neutral","Agree","Yes","Very Important","Important","Moderately Important","Very Important","Very Important","Very Important","Very Important","Important","Yes","nil"],
["4","22","Male","Single","Student","Below Rs.10000","Post Graduate","4","12.985","77.5533","560010","Walk-in"," Direct call","Lunch"," Dinner","Non Veg foods (Lunch / Dinner)"," Veg foods (Breakfast / Lunch / Dinner)","Agree","Agree","Agree","Agree","Agree","Neutral","Neutral","Agree","Strongly agree","Strongly agree","Agree","Strongly agree","Agree","Disagree","Strongly agree","Strongly agree","Neutral","Neutral","Neutral","Disagree","Yes","Weekend (Sat & Sun)","30 minutes","Agree","Agree","Agree","Agree","Agree","Yes","Important","Important","Moderately Important","Important","Important","Important","Very Important","Very Important","Yes","NIL"]],"shape":{"columns":55,"rows":5}}

### Insights

We can understand that there are more categorical variables and very less continuous variables There is also longitude and latitude variable which can help us for geospatial analysis Most of the variables has likert scale classes inside the variable , it needs to be encoded

## Step-3 Analyze the dataset

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 388 entries, 0 to 387
Data columns (total 55 columns):
 #   Column                                        Non-Null Count  Dtype

---  ------                                        --------------  -----

 0   Age                                           388 non-null    int64

 1   Gender                                        388 non-null    object

 2   Marital Status                                388 non-null    object

 3   Occupation                                    388 non-null    object
```

```
 4   Monthly Income                          388 non-null    object

 5   Educational Qualifications              388 non-null    object

 6   Family size                             388 non-null    int64

 7   latitude                                388 non-null
float64
 8   longitude                               388 non-null
float64
 9   Pin code                                388 non-null    int64

 10  Medium (P1)                             388 non-null    object

 11  Medium (P2)                             388 non-null    object

 12  Meal(P1)                                388 non-null    object

 13  Meal(P2)                                388 non-null    object

 14  Perference(P1)                          388 non-null    object

 15  Perference(P2)                          388 non-null    object

 16  Ease and convenient                     388 non-null    object

 17  Time saving                             388 non-null    object

 18  More restaurant choices                 388 non-null    object

 19  Easy Payment option                     388 non-null    object

 20  More Offers and Discount                388 non-null    object

 21  Good Food quality                       388 non-null    object

 22  Good Tracking system                    388 non-null    object

 23  Self Cooking                            388 non-null    object

 24  Health Concern                          388 non-null    object

 25  Late Delivery                           388 non-null    object

 26  Poor Hygiene                            388 non-null    object

 27  Bad past experience                     388 non-null    object
```

```
 52  Good Quantity                                 388 non-null    object

 53  Output                                        388 non-null    object

 54  Reviews                                       387 non-null    object

dtypes: float64(2), int64(3), object(50)
memory usage: 166.8+ KB
```

*Content of dataset*

The dataset has nearly 55 variables based on the following titles

Demographics of consumers Overall/general purchase decision Time of delivery influencing the purchase decision Rating of Restaurant influencing the purchase decision For more information on each columns check out the description provided below head column title

```
data.describe()
```


```
{"columns":[{"name":"index","rawType":"object","type":"string"},
{"name":"Age","rawType":"float64","type":"float"},{"name":"Family
size","rawType":"float64","type":"float"},
{"name":"latitude","rawType":"float64","type":"float"},
{"name":"longitude","rawType":"float64","type":"float"},{"name":"Pin
code","rawType":"float64","type":"float"}],"ref":"56432554-01bf-4e76-
943d-6074b753a1aa","rows":
[["count","388.0","388.0","388.0","388.0","388.0"],
["mean","24.628865979381445","3.2809278350515463","12.97205798969072",
"77.60015953608249","560040.1134020619"],
["std","2.9755926606729046","1.3510249396453136","0.044489248628106076
","0.05135391701271844","31.399608710233185"],
["min","18.0","1.0","12.8652","77.4842","560001.0"],
["25%","23.0","2.0","12.9369","77.565275","560010.75"],
["50%","24.0","3.0","12.977","77.5921","560033.5"],
["75%","26.0","4.0","12.997025","77.6309","560068.0"],
["max","33.0","6.0","13.102","77.7582","560109.0"]],"shape":
{"columns":5,"rows":8}}
```

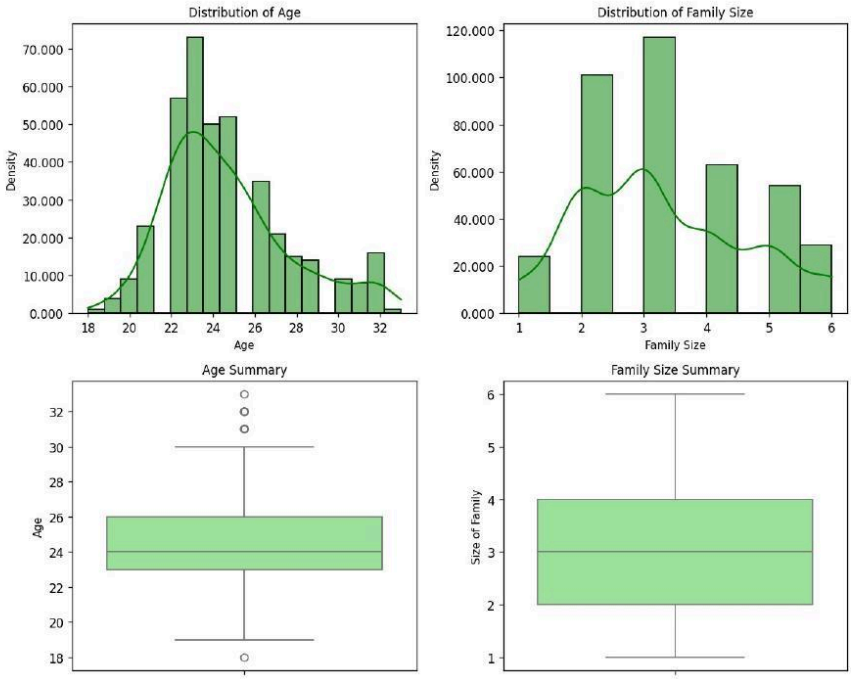
```
","False","False","False","False","False","False","False","False","Fal
se","False","False","False","False","False","False","False","False","F
alse","False","False","False","False","False","False","False","False",
"False","False","False","False","False","False","False","False","False
","False","False","False"],
["47","False","False","False","False","False","False","False","False",
"False","False","False","False","False","False","False","False","False
","False","False","False","False","False","False","False","False","Fal
se","False","False","False","False","False","False","False","False","F
alse","False","False","False","False","False","False","False","False",
"False","False","False","False","False","False","False","False","False
","False","False","False"],
["48","False","False","False","False","False","False","False","False",
"False","False","False","False","False","False","False","False","False
","False","False","False","False","False","False","False","False","Fal
se","False","False","False","False","False","False","False","False","F
alse","False","False","False","False","False","False","False","False",
"False","False","False","False","False","False","False","False","False
","False","False","False"],
["49","False","False","False","False","False","False","False","False",
"False","False","False","False","False","False","False","False","False
","False","False","False","False","False","False","False","False","Fal
se","False","False","False","False","False","False","False","False","F
alse","False","False","False","False","False","False","False","False",
"False","False","False","False","False","False","False","False","False
","False","False","False"]],"shape":{"columns":55,"rows":388}}
```


### Demographic analysis 1

```
#Pivot table
delivery_pivot1=pd.pivot_table(data,index=["Gender","Marital Status"],
                               values=['Age','Family size'],
                               #aggfunc=[np.mean,len]
                               aggfunc = ['mean'], margins=True)

#Adding color gradient
cm = sns.light_palette("green", as_cmap=True)
delivery_pivot1.style.background_gradient(cmap=cm)
```

```
<pandas.io.formats.style.Styler at 0x16cfb244280>
```




## Geospatial Analysis

### Geo Painter

```
Age_band = data[(data.Age.isin(range(18,40)))]
# Creating a map
m_2 = folium.Map(location=[12.9716,77.5946], tiles='cartodbpositron',
zoom_start=13)

# Adding points to the map
for idx, row in Age_band.iterrows():
    Marker([row['latitude'], row['longitude']]).add_to(m_2)

# Displaying the map
m_2
```

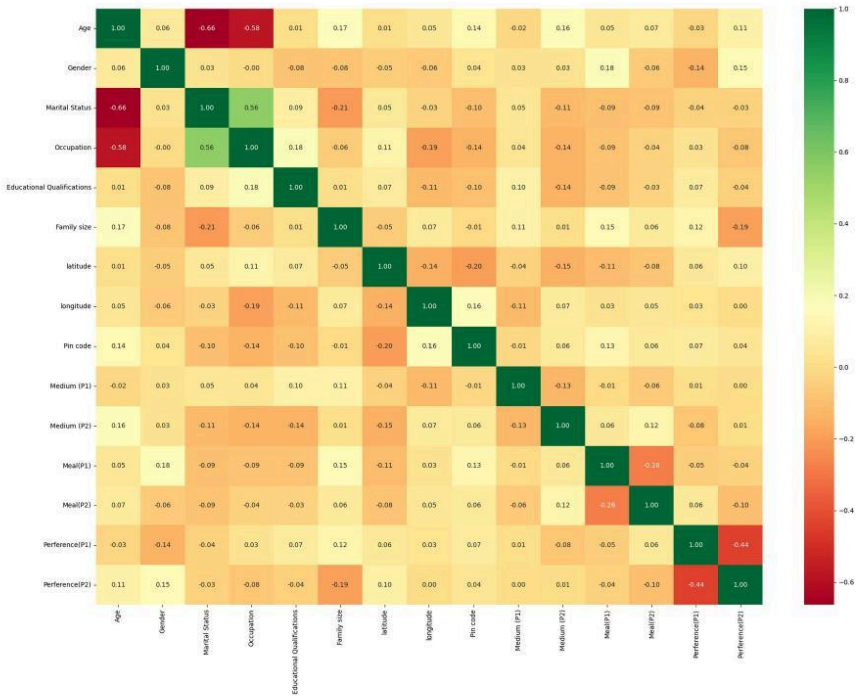
```
<folium.folium.Map at 0x16ca009f940>
```

### Feature Correlation

```
# Change some features value to number, so we can plot it
data = data.replace(['0', '<10k', '10k-25k', '25k-50k', '>50k'],
[1,2,3,4,5])
```

```
data = data.replace(['15 minutes', '30 minutes', '45 minutes', '60
minutes', 'more than 60 minutes'], [1,2,3,4,5])
trans_col = ['Gender', 'Marital Status', 'Occupation','Educational
Qualifications','Medium (P1)',
             'Medium (P2)', 'Meal(P1)', 'Meal(P2)', 'Perference(P1)',
'Perference(P2)']
for col in trans_col:
    data[col] = LabelEncoder().fit_transform(data[col])

plt.figure(figsize=(20,15))
# The fix: Add numeric_only=True
corr_matrix = data.corr(method='pearson', numeric_only=True)

# Generate the heatmap with annotations and a nice color map
sns.heatmap(corr_matrix, annot=True, cmap='RdYlGn', fmt='.2f')
plt.tight_layout()
plt.show()
```

```
<Figure size 2000x1500 with 0 Axes>

<Figure size 2000x1500 with 0 Axes>

<Figure size 2000x1500 with 0 Axes>
```